

EXPERIMENT 11 – DATA-DRIVEN AUTOMATION USING PYTHON BEHAVE FRAMEWORK

1. Module Name and Experiment Title

Module Name: Module 3 – Python BDD RESTful Automations

Experiment Number: 11

Experiment Title: Data-Driven Automation Using Python Behave Framework

Programming Language: Python

Automation Tool: Selenium WebDriver

BDD Framework: Behave

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

Automating login functionality using a single set of credentials is insufficient to verify how an application responds to different login scenarios.

The objective of this experiment is to implement data-driven testing using the Python Behave framework and Selenium WebDriver. Test data is stored in a CSV file and used to execute multiple login scenarios automatically.

The automation must verify successful login, invalid credentials, and a locked-out user. It should compare the actual results with the expected results and report whether each test case passes or fails.

3. Objective

The main objectives of this experiment are:

1. To understand data-driven testing using the Python Behave framework.
2. To automate the login functionality of the SauceDemo website using Selenium WebDriver.
3. To store multiple sets of test data in a CSV file.
4. To execute multiple login test cases using a single Behave scenario.
5. To verify successful and unsuccessful login attempts.
6. To compare actual results with expected results.
7. To capture screenshots of important stages during test execution.
8. To generate a summary of the test execution results.

4. Tools, Software and Concepts Used

Tools and Software

Tool/Software	Purpose
Python	Programming language used to implement the automation
Selenium WebDriver	Automates browser interactions
Behave	Executes BDD scenarios and step definitions
Google Chrome	Browser used for automation
Visual Studio Code	Code editor used to develop and execute the project
CSV	Stores multiple sets of login test data

Concepts Used

- **Data-Driven Testing (DDT):** Executes the same test logic using multiple sets of input data.
- **Behavior-Driven Development (BDD):** Describes test scenarios in a human-readable Given-When-Then format.
- **Selenium WebDriver:** Locates elements, enters credentials, clicks buttons, and verifies page content.
- **CSV File Handling:** Reads test data from a CSV file.
- **Assertions:** Checks whether the actual application behavior matches the expected result.
- **Explicit Waits:** Waits for web elements to become available before interacting with them.
- **Screenshot Capture:** Saves screenshots of important stages of the test execution.

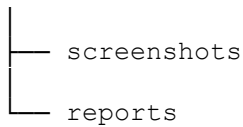
5. Source Code / Implementation Steps

5.1 Project Folder Structure

The project is organized as follows:

Experiment 11 - Data-Driven Automation

```
|
|— features
|   |— login.feature
|   |— environment.py
|   |
|   |— steps
|       |— login_steps.py
|
|— test_data
    |— login_data.csv
```



5.2 Test Data

The login test data is stored in `login_data.csv`. It contains three test cases with different usernames and their expected results.

Test Case	Username	Password	Expected Result
1	standard_user	secret_sauce	success
2	invalid_user	secret_sauce	error
3	locked_out_user	secret_sauce	error

The CSV file allows the same login automation to be executed with different input data without changing the test logic.

5.3 Feature File

File: `features/login.feature`

The feature file defines the login scenario using Gherkin syntax.

Feature: Data-Driven SauceDemo Login

```
Scenario: Login using multiple sets of test data
  Given the user opens the SauceDemo website
  When the user performs login tests using the CSV data
  Then all login test cases should produce the expected results
```

Explanation:

- **Feature:** Describes the login functionality being tested.
- **Scenario:** Defines the execution of multiple login test cases.
- **Given:** Opens the SauceDemo website.
- **When:** Executes login tests using data from the CSV file.
- **Then:** Verifies that all test cases produce their expected results.

5.4 Environment Configuration

File: `features/environment.py`

The environment file manages browser setup and cleanup for the Behave scenario.

It initializes the Chrome WebDriver before the scenario starts and maximizes the browser window. After the scenario finishes, it closes the browser using `driver.quit()`.

This ensures that the browser is automatically closed after test execution.

5.5 Step Definitions

File: `features/steps/login_steps.py`

The step definition file contains the Selenium automation logic.

The implementation performs the following operations:

1. Opens the SauceDemo website using Selenium WebDriver.
2. Captures a screenshot of the initial login page.
3. Reads the login test data from `login_data.csv`.
4. Enters each username and password into the login form.
5. Clicks the Login button.
6. Verifies the actual result against the expected result.
7. Captures screenshots of the credentials, successful login page, and error messages.
8. Prints the result of each test case.
9. Displays a summary of the data-driven test execution.

For the `standard_user` test case, the automation verifies that the user is redirected to the Products page.

For the `invalid_user` and `locked_out_user` test cases, the automation verifies that the appropriate login error is displayed.

The implementation uses Selenium WebDriver and Behave step definitions to execute the test cases.

5.6 Installation and Execution

Install the required Python packages using:

```
pip install selenium behave
```

Navigate to the Experiment 11 project directory in the terminal and execute:

```
behave --no-capture
```

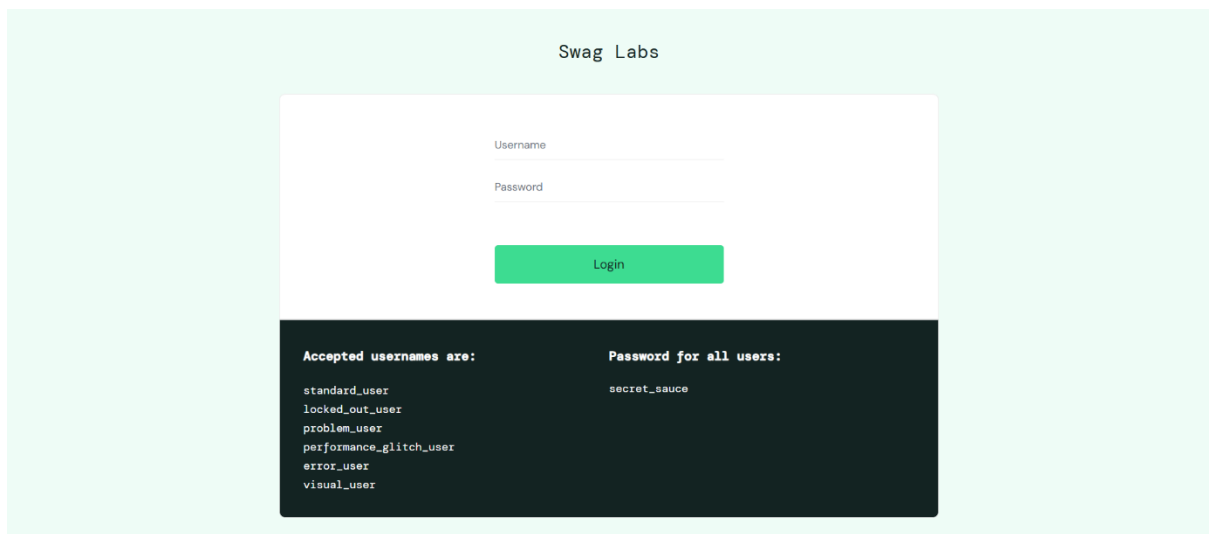
The `--no-capture` option displays the output generated by the step definitions in the terminal.

6. Output with Clearly Labelled Screenshots

The experiment was executed successfully using three sets of login test data. The following screenshots document the execution.

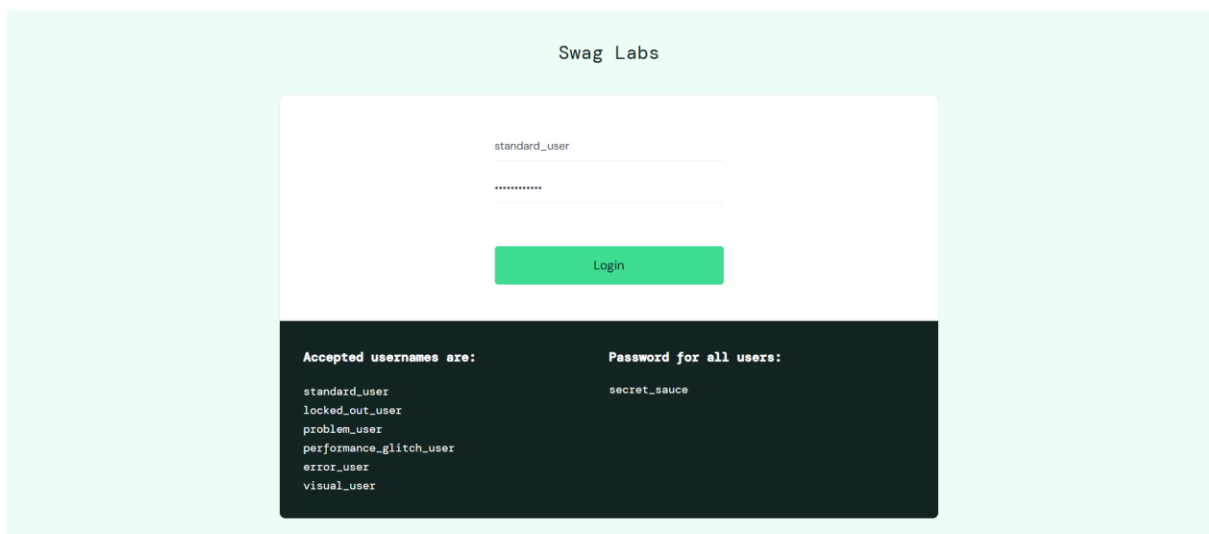
Screenshot 1: Initial SauceDemo Login Page

Description: The initial SauceDemo login page is displayed before entering any credentials.



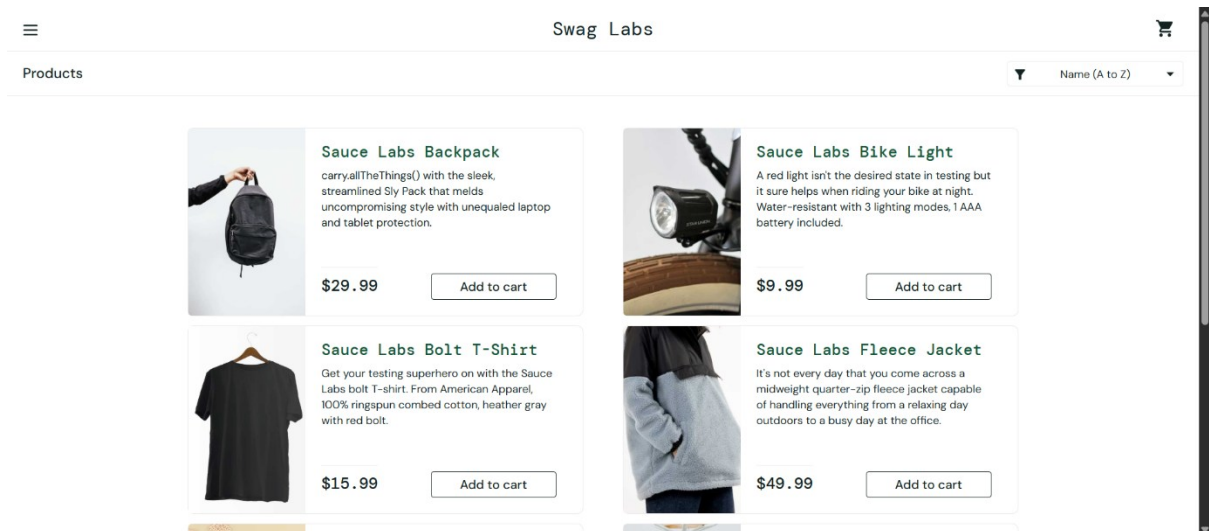
Screenshot 2: Standard User Credentials

Description: The automation enters the `standard_user` username and password into the login form.



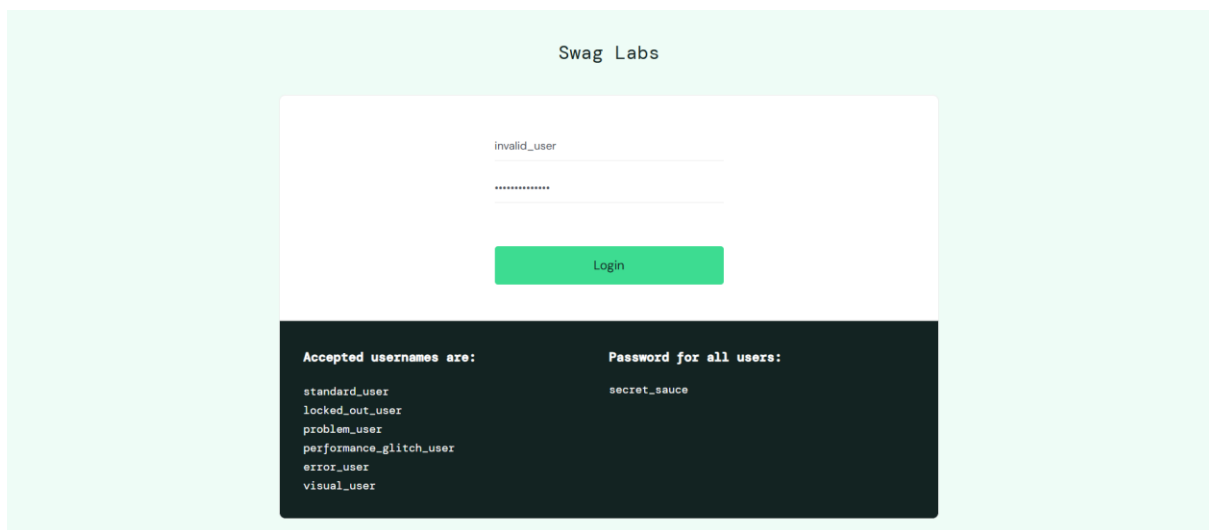
Screenshot 3: Successful Login – Products Page

Description: After submitting valid credentials, the user is successfully redirected to the Products page. The automation verifies the successful login.



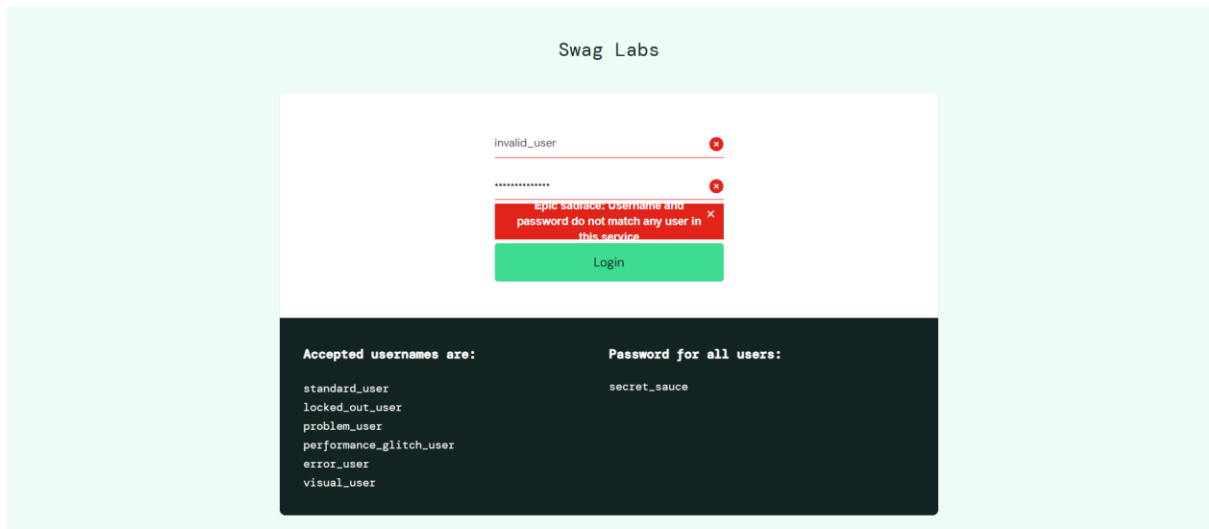
Screenshot 4: Invalid User Credentials

Description: The automation enters the invalid username `invalid_user` and the password before submitting the login form.



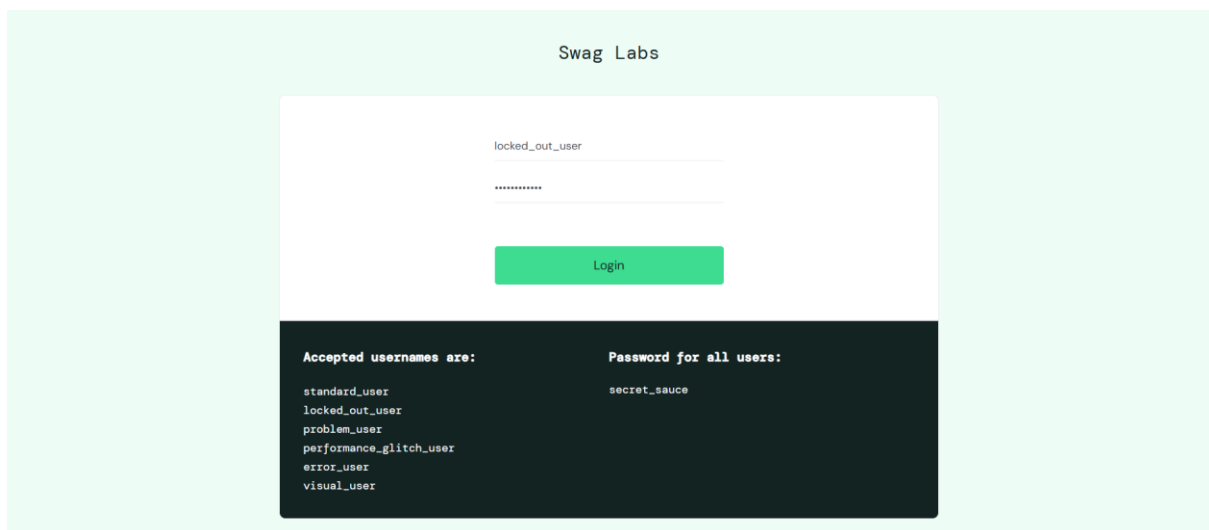
Screenshot 5: Invalid Login Error Message

Description: The website displays an error message indicating that the username and password do not match any user. The automation verifies the expected error result.



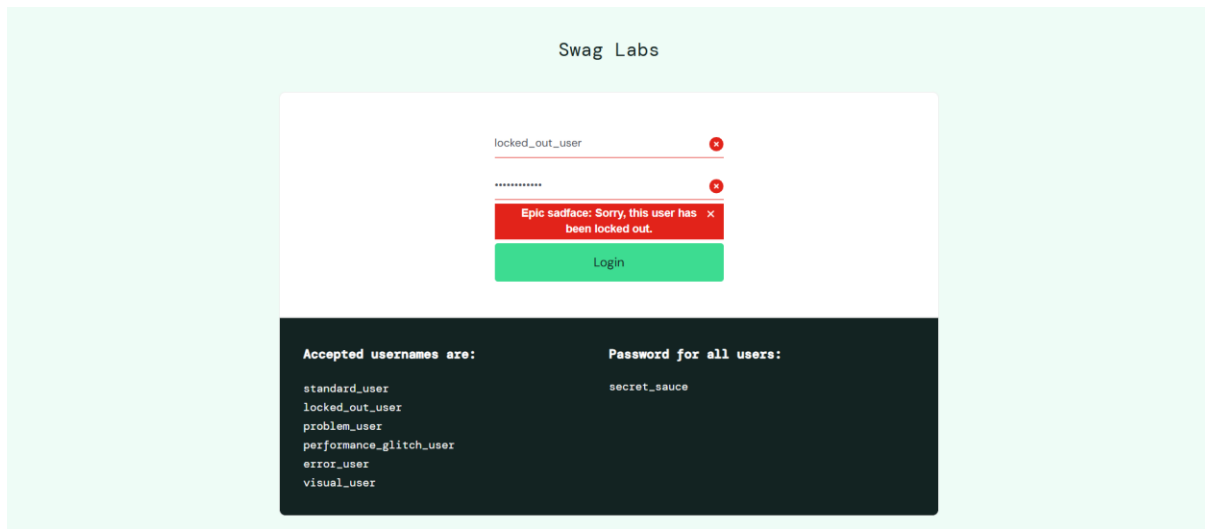
Screenshot 6: Locked-Out User Credentials

Description: The automation enters the `locked_out_user` username and password into the login form.



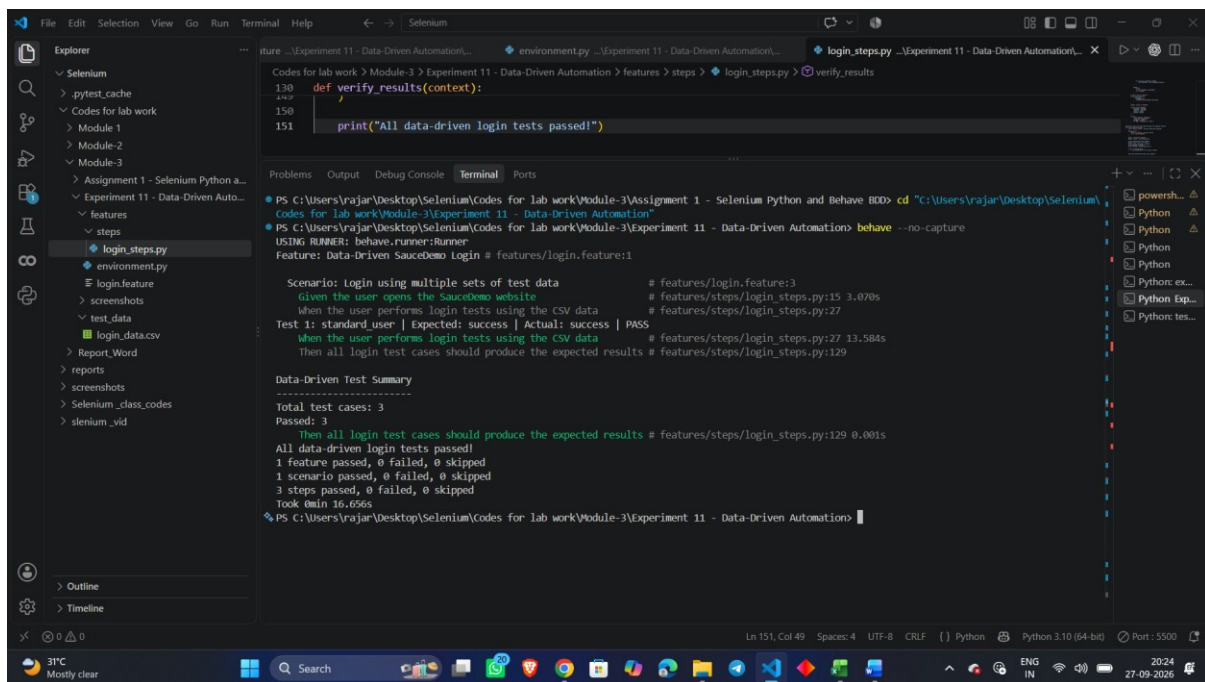
Screenshot 7: Locked-Out User Error Message

Description: The website displays an error message stating that the user has been locked out. The automation verifies the expected error result.



Screenshot 8: Terminal Output – Successful Execution

Description: The terminal displays the Behave execution summary. All three data-driven login test cases passed successfully, with zero failures and zero skipped tests.



Execution Summary

Data-Driven Test Summary

Total test cases: 3

Passed: 3

All data-driven login tests passed!

1 feature passed, 0 failed, 0 skipped
 1 scenario passed, 0 failed, 0 skipped
 3 steps passed, 0 failed, 0 skipped

7. Result, Observation and Conclusion

Result

The Data-Driven Automation experiment was successfully implemented using Python, Selenium WebDriver, and the Behave framework.

Three login test cases were executed using data stored in a CSV file:

- Successful login using `standard_user`.
- Unsuccessful login using `invalid_user`.
- Unsuccessful login using `locked_out_user`.

All three test cases passed, and the automation successfully verified the expected application behavior.

Observation

1. The Behave framework executed the login scenario using Gherkin syntax.
2. The CSV file provided multiple sets of test data for the same automation logic.
3. Selenium WebDriver automated the login process and verified the resulting pages and error messages.
4. The application displayed the expected result for each test case.
5. Screenshots provided visual evidence of the login process and its outcomes.
6. The terminal displayed the execution summary, confirming that all three test cases passed.

Conclusion

This experiment demonstrated how data-driven testing can be implemented using the Python Behave framework and Selenium WebDriver. By storing test data in a CSV file, multiple login scenarios were executed using a single scenario and reusable step definitions.

The experiment successfully verified valid login, invalid credentials, and locked-out user behavior. It also demonstrated how automated assertions, screenshots, and execution summaries can be used to validate and document web application testing.